

МРС-63 — Добавить

CollectProductPricesUseCase и выбор товаров по oldest-first

Summary

Реализовать отдельный рабочий сценарий сбора цен из постоянного каталога `product_catalog`.

В рамках задачи необходимо добавить:

CollectProductPricesUseCase

Use case должен:

1. создать `crawler_run`;
2. создать `ingestion_run`;
3. выбрать из `product_catalog` ограниченную порцию активных товаров;
4. выбирать в первую очередь товары, которые никогда не проверялись;
5. затем выбирать товары с самым старым `last_checked_at`;
6. поставить выбранные URL в существующую `price_collect_queue`;
7. обработать очередь через существующий extractor;
8. сохранить `product` и `price_snapshot`;
9. после успеха обновить состояние записи `product_catalog`;
10. после ошибки обновить счётчик ошибок и `next_check_at`;
11. завершить `ingestion_run` и `crawler_run`;
12. не выполнять discovery;
13. не обходить category seed;
14. не изменять каталог через discovery upsert.

После выполнения задачи проект должен иметь первый практически рабочий режим:

```
product_catalog
→ oldest-first selection
→ price_collect_queue
→ extractor
→ product / price_snapshot
→ product_catalog scheduling state update
```

Context

Репозиторий:

<https://github.com/bodomus/PriceCrawler>

Базовая ветка:

MPC-62

Создать отдельную рабочую ветку:

Codex/MPC-63

Dependencies

Задача зависит от завершённых:

MPC-61 — постоянный каталог `product_catalog`
MPC-62 — `RefreshProductCatalogUseCase`

До начала реализации убедиться, что в базовой ветке существуют:

- таблица `product_catalog` ;
- `ProductCatalogItem` ;
- `ProductCatalogUpsertItem` ;
- `IProductCatalogRepository` ;
- `PgProductCatalogRepository` ;
- `RefreshProductCatalogUseCase` ;
- отдельная Worker-команда `catalog-refresh` ;
- разделённая семантика:
- `MaxUrls` — catalog discovery;
- `MaxProductsPerRun` — price collection;
- существующая `price_collect_queue` ;
- существующий `IProductCardExtractor` ;
- существующий `IPriceSnapshotRepository` ;
- `ICrawlerRunRepository` ;
- `IIngestionRunRepository` .

Если обязательных компонентов нет, не создавать параллельную альтернативную архитектуру.
Зафиксировать блокирующую зависимость.

Current behavior

После MPC-62 проект умеет:

```
CategorySeeds
→ discovery
→ product_catalog
```

Но постоянный каталог пока не используется для ежедневного сбора цен.

Существующий `RunCrawlerUseCase` выполняет:

```
discovery
→ queue
→ extraction
→ snapshots
```

В рамках MPC-63 нужно добавить новый use case, который не запускает discovery и использует сохранённый каталог.

Goal

Добавить самостоятельный application flow:

```
CollectProductPricesUseCase
↓
IProductCatalogRepository.GetDueProductsAsync
↓
price_collect_queue
↓
IProductCardExtractor
↓
IPriceSnapshotRepository
↓
IProductCatalogRepository.MarkChecked / MarkFailed
```

Use case должен быть пригоден для ежедневного запуска.

Required architecture

Соблюдать текущие слои:

```
Domain
Application
Infrastructure
Worker
```

Domain

Domain должен содержать:

- repository contracts;
- immutable models;
- enums;
- value objects.

Domain не должен зависеть от:

- PostgreSQL;
- Npgsql;
- EF Core;
- Infrastructure;
- Application;
- Worker;
- Web.

Application

Application должен содержать:

- `CollectProductPricesUseCase`;
- orchestration logic;
- retry decision integration;
- result model;
- scheduling calculations, если они не относятся к SQL.

Application не должен содержать SQL.

Infrastructure

Infrastructure должен содержать:

- PostgreSQL repository implementation;
- DB routines;
- batch operations;
- row mapping.

Worker

Worker должен только выбирать use case по команде и запускать его.

Не помещать бизнес-логику oldest-first в `Program.cs`.

Scope

1. Расширить `IProductCatalogRepository`

Добавить методы:

```
Task<IReadOnlyList<ProductCatalogItem>> GetDueProductsAsync(
    int limit,
    DateTimeOffset nowUtc,
    CancellationToken ct);

Task MarkCheckedAsync(
    long catalogItemId,
    DateTimeOffset checkedAtUtc,
    DateTimeOffset nextCheckAtUtc,
    string? externalId,
    string? slug,
    CancellationToken ct);

Task MarkFailedAsync(
    long catalogItemId,
    DateTimeOffset attemptedAtUtc,
    DateTimeOffset nextCheckAtUtc,
    CancellationToken ct);
```

Допускается batch API вместо двух отдельных методов, если он реализован корректно и уменьшает количество DB round-trip.

Предпочтительный production-вариант:

```
Task MarkCheckedBatchAsync(
    IReadOnlyCollection<ProductCatalogCheckSuccess> items,
    CancellationToken ct);

Task MarkFailedBatchAsync(
    IReadOnlyCollection<ProductCatalogCheckFailure> items,
    CancellationToken ct);
```

Однако не усложнять реализацию только ради batching, если текущий queue pipeline обновляет элементы по одному.

Главное требование:

- выбор oldest-first выполняется атомарно и детерминированно;
- состояние каталога обновляется после каждого обработанного товара либо надёжным batch-механизмом.

2. Добавить модели результата обновления каталога

При необходимости добавить immutable модели:

```
public sealed record ProductCatalogCheckSuccess(  
    long CatalogItemId,  
    DateTimeOffset CheckedAtUtc,  
    DateTimeOffset NextCheckAtUtc,  
    string? ExternalId,  
    string? Slug);
```

```
public sealed record ProductCatalogCheckFailure(  
    long CatalogItemId,  
    DateTimeOffset AttemptedAtUtc,  
    DateTimeOffset NextCheckAtUtc);
```

Не помещать SQL-specific типы в Domain.

3. Реализовать oldest-first selection

Добавить PostgreSQL routine:

```
product_catalog_get_due
```

или эквивалентное имя в существующем стиле.

Она должна выбирать только записи:

```
where is_active = true  
and (  
    next_check_at is null  
    or next_check_at <= p_now  
)
```

Порядок:

```
order by  
    last_checked_at nulls first,  
    next_check_at nulls first,  
    id  
limit p_limit
```

Обязательная семантика:

1. сначала товары с `last_checked_at is null`;
2. затем товар с самой старой `last_checked_at`;
3. при одинаковых значениях — стабильный порядок по `id`.

Не использовать случайную сортировку.

Не сортировать по URL.

Не выбирать inactive товары.

4. Защита от повторного выбора

Одновременные запуски не должны выбирать одну и ту же порцию товаров.

Предпочтительный вариант — добавить lease-поля в `product_catalog`:

```
reserved_at    timestampz,  
reserved_until timestampz,  
reserved_by    varchar(200)
```

Если MPC-61 уже предусматривала другое решение, следовать существующему дизайну.

Required behavior

Выбор должен:

1. найти due rows;
2. зарезервировать их;
3. вернуть выбранные записи;
4. выполнить это в одной транзакции или одной DB routine;
5. использовать:

```
for update skip locked
```

или эквивалентный PostgreSQL-механизм.

Запрещена схема:

```
SELECT due rows  
→ вернуть в C#  
→ отдельным UPDATE выставить reservation
```

между этими шагами возможна гонка.

5. Lease semantics

Добавить настройку:

```
"Crawler": {  
  "CatalogLeaseSeconds": 1800  
}
```

Допустимо разместить её в `Queue`, если это лучше соответствует текущей конфигурации. Выбрать один раздел и документировать.

Рекомендуемое значение по умолчанию:

```
1800 секунд
```

При резервировании:

```
reserved_at = now  
reserved_until = now + lease  
reserved_by = worker id
```

Записи с истёкшим lease снова доступны для выбора.

После успешной или окончательно неуспешной обработки reservation должна очищаться.

6. Добавить result model use case

Добавить:

```
public sealed record CollectProductPricesResult(  
    long RunId,  
    string Status,  
    int SelectedCount,  
    int EnqueuedCount,  
    int SucceededCount,  
    int FailedCount,  
    int RetryCount,  
    int DeadCount,  
    string? ErrorCode,  
    string? Message);
```


Семантика

SelectedCount

Сколько записей выбрано из `product_catalog`.

EnqueuedCount

Сколько записей реально добавлено в `price_collect_queue`.

SucceededCount

Сколько товаров успешно обработано и для которых сохранено наблюдение.

FailedCount

Количество обработок, завершившихся ошибкой в рамках текущего запуска.

RetryCount

Количество queue items, находящихся в retry-состоянии в финальной статистике.

DeadCount

Количество окончательно неуспешных queue items.

Status

ok
error

Правило:

DeadCount > 0
→ error

Если выбранных товаров нет, см. отдельный раздел ниже.

7. Добавить интерфейс use case

```
public interface ICollectProductPricesUseCase
{
    Task<CollectProductPricesResult> ExecuteAsync(
        CancellationToken ct);
}
```

8. Добавить `CollectProductPricesUseCase`

Рекомендуемые зависимости:

```
IOptions<CrawlerOptions>
IOptions<QueueOptions>
IProductCatalogRepository
IProductCardExtractor
ICrawlerRunRepository
IIgestionRunRepository
IPriceCollectQueueRepository
IPriceSnapshotRepository
ILogger<CollectProductPricesUseCase>
```

Допускается вынести общий queue-drain код из `RunCrawlerUseCase` в отдельный application service.

Предпочтительный вариант:

```
IPriceCollectionQueueProcessor
```

или:

```
PriceCollectionQueueProcessor
```

Он должен инкапсулировать:

- reserve queue batch;
- extractor call;
- snapshot persistence;
- retry/dead logic;
- queue status updates;
- catalog status updates.

Не копировать сотни строк `RunCrawlerUseCase` без необходимости.

Но также не выполнять масштабный unrelated refactoring.

Required execution flow

Шаг 1. Создать `crawler_run`

Создать до выбора товаров:

```
var runId = await crawlerRunRepository.StartAsync(
    "price-collection",
    ct);
```

Допустимый source:

```
price-collection
```

Предпочтительно использовать именно его.

Шаг 2. Создать ingestion_run

```
var ingestionRunId = await ingestionRunRepository.StartAsync(
    runId,
    ct);
```

Сбор цен является ingestion flow, поэтому ingestion_run обязателен.

Шаг 3. Выбрать due products

Вызвать:

```
var selected = await productCatalogRepository.GetDueProductsAsync(
    limit,
    nowUtc,
    ct);
```

Лимит:

```
Math.Max(1, options.Value.MaxProductsPerRun)
```

MaxUrls здесь не использовать.

Шаг 4. Empty selection

Если due products отсутствуют:

1. не создавать queue items;
2. не вызывать extractor;
3. завершить ingestion_run со статусом ok;

4. завершить `crawler_run` со статусом `ok`;

5. вернуть:

```
SelectedCount = 0
EnqueuedCount = 0
SucceededCount = 0
FailedCount = 0
RetryCount = 0
DeadCount = 0
Status = ok
```

Note:

```
no due catalog products
```

Пустой due batch не является ошибкой.

Шаг 5. Поставить товары в очередь

Для каждого выбранного catalog item создать queue item.

Idempotency key должна включать:

```
runId
catalogItemId
normalizedUrl
```

Пример:

```
sha256("${runId}:{catalogItemId}:{normalizedUrl}")
```

Нужно обеспечить связь queue item с catalog item.

Required schema change for queue linkage

Добавить в `price_collect_queue`:

```
product_catalog_id bigint null
```

Foreign key:

```
references product_catalog(id)
```

Политика удаления:

```
on delete restrict
```

или:

```
on delete set null
```

Предпочтительно:

```
on delete restrict
```

так как записи каталога не должны физически удаляться.

Добавить индекс:

```
create index if not exists ix_price_collect_queue_product_catalog_id  
on price_collect_queue(product_catalog_id);
```

Обновить enqueue routine так, чтобы она принимала catalog IDs вместе с URL и idempotency keys.

Порядок массивов должен проверяться:

```
urls.Length == idempotencyKeys.Length == catalogIds.Length
```

Несовпадение размеров должно завершать routine ошибкой.

Шаг 6. Queue processing

Использовать существующий flow:

```
ReserveBatchAsync  
→ ExtractAsync  
→ StoreObservationAsync  
→ MarkSucceeded / MarkRetry / MarkDead
```

Но добавить обновление `product_catalog`.

Success behavior

После успешного сохранения observation:

1. queue item помечается succeeded;
2. `product_catalog.last_checked_at` обновляется;
3. `product_catalog.next_check_at` выставляется;
4. `consecutive_errors` сбрасывается в `0`;
5. reservation очищается;
6. `external_id` обновляется, если extractor вернул непустое значение;
7. `slug` обновляется, если extractor вернул непустое значение.

Required next check value

Добавить настройку:

```
"Crawler": {  
  "SuccessfulCheckIntervalHours": 24  
}
```

Значение по умолчанию:

24

Расчёт:

```
nextCheckAtUtc = checkedAtUtc.AddHours(  
  Math.Max(1, options.Value.SuccessfulCheckIntervalHours));
```

Не хардкодить 24 часа внутри SQL.

Failure behavior

Transient failure with queue retry

Если queue item переводится в retry:

- `last_checked_at` пока не обновлять;
- `consecutive_errors` пока не увеличивать;
- reservation каталога сохранить или продлить так, чтобы другой запуск не выбрал тот же товар;
- `next_check_at` не использовать для queue-level retry;
- повторная попытка должна происходить через существующую очередь.

Queue retry остаётся ответственностью `price_collect_queue`.

Final dead failure

Когда item окончательно переходит в dead:

1. увеличить `consecutive_errors` на `1`;
2. установить `last_checked_at = attemptedAtUtc`;
3. установить `next_check_at` по backoff;
4. очистить reservation;
5. сохранить crawl error;
6. пометить queue item dead.

Catalog failure backoff

Добавить настройки:

```
"Crawler": {  
  "CatalogFailureBaseDelayMinutes": 60,  
  "CatalogFailureMaxDelayHours": 24  
}
```

Backoff:

```
base × 2(consecutive_errors before increment)
```

Ограничить максимальным значением.

Пример:

```
ошибка 1 → 60 минут  
ошибка 2 → 120 минут  
ошибка 3 → 240 минут  
...  
максимум → 24 часа
```

Защититься от overflow.

Расчёт должен быть покрыт unit tests.

Допускается отдельный stateless helper:

ProductCatalogRetryPolicy

Required atomicity

Для успешного товара критические операции:

```
price_observation_store  
queue mark succeeded  
product_catalog mark checked
```

должны иметь согласованное поведение.

Предпочтительный вариант — одна PostgreSQL routine, которая:

1. сохраняет observation;
2. помечает queue item succeeded;
3. обновляет product catalog;
4. выполняется транзакционно.

Например:

```
price_observation_store_and_complete
```

Если это слишком большой рефакторинг, допускаются отдельные routine calls, но тогда необходимо:

- документировать риск частичного сохранения;
- обеспечить idempotency повторного выполнения;
- покрыть сценарий тестом.

Предпочтение отдать транзакционной DB routine.

Аналогично для final failure предпочтительно одной routine:

```
price_collect_item_mark_dead_and_catalog_failed
```

Не использовать распределённую транзакцию на стороне C#.

product_catalog update routines

Добавить либо обновить routines:


```
product_catalog_mark_checked  
product_catalog_mark_failed  
product_catalog_release_reservation
```

product_catalog_mark_checked

Должна:

```
last_checked_at = checked_at  
next_check_at = next_check_at  
consecutive_errors = 0  
external_id = non-empty incoming value, иначе сохранить старое  
slug = non-empty incoming value, иначе сохранить старое  
reserved_at = null  
reserved_until = null  
reserved_by = null  
updated_at = now()
```

product_catalog_mark_failed

Должна:

```
last_checked_at = attempted_at  
next_check_at = calculated/provided next_check_at  
consecutive_errors = consecutive_errors + 1  
reserved_at = null  
reserved_until = null  
reserved_by = null  
updated_at = now()
```

Routine должна возвращать новое значение `consecutive_errors`, если оно требуется для расчётов.

Cancellation behavior

При `OperationCanceledException`:

1. не возвращать успешный result;
2. попытаться завершить `ingestion_run` со статусом error;
3. попытаться завершить `crawler_run` со статусом error;
4. освободить только reservation, относящиеся к текущему run/worker, если это безопасно;
5. повторно выбросить исключение.

Если cleanup вызывается с отменённым token, использовать отдельный короткоживущий token только если подобный подход согласован с проектом.

Не выполнять длительный cleanup.

Unexpected top-level error

При ошибке до завершения run:

```
error code = price_collection_failed
```

Нужно:

- завершить ingestion run;
- завершить crawler run;
- вернуть error result либо пробросить исключение в соответствии с текущим application contract.

Предпочтительно следовать стилю `RunCrawlerUseCase`, но не скрывать ошибку без логирования.

Refactoring existing `RunCrawlerUseCase`

Цель MPC-63 — новый рабочий use case.

Допускается:

- извлечь queue processing в общий service;
- переиспользовать retry policy;
- переиспользовать normalization helpers;
- переиспользовать worker ID generation.

Не допускается:

- удалить старый `RunCrawlerUseCase`;
- изменить его публичное поведение без тестов;
- переключить default Worker mode на новый use case без явной команды;
- переписать discovery.

Старый use case остаётся legacy/debug flow до отдельного тикета.

Worker command

Добавить команду:

```
dotnet run --project VarPrice.Worker -- collect-prices
```

Поведение:

```
collect-prices
→ выполнить только CollectProductPricesUseCase
→ завершить процесс
```

Не запускать:

- catalog refresh;
- category discovery;
- sitemap discovery;
- старый `RunCrawlerUseCase`.

Существующая команда:

```
catalog-refresh
```

должна продолжать работать.

Optional convenience command

Допускается добавить:

```
dotnet run --project VarPrice.Worker -- run-all
```

Flow:

```
catalog-refresh
→ collect-prices
```

Но только если реализация минимальна и не усложняет scope.

`run-all` не является обязательным acceptance criterion.

Configuration

Добавить настройки:

```
{
  "Crawler": {
    "MaxProductsPerRun": 5000,
    "MaxConcurrency": 2,
    "RequestsPerSecond": 1.0,

    "CatalogLeaseSeconds": 1800,
    "SuccessfulCheckIntervalHours": 24,
    "CatalogFailureBaseDelayMinutes": 60,
    "CatalogFailureMaxDelayHours": 24
  }
}
```

Не обязательно менять production default `MaxProductsPerRun` на 5000 в рамках задачи.

Но документация должна показать рабочий пример.

Validation

Значения меньше допустимого минимума должны безопасно нормализоваться:

```
MaxProductsPerRun >= 1
CatalogLeaseSeconds >= 30
SuccessfulCheckIntervalHours >= 1
CatalogFailureBaseDelayMinutes >= 1
CatalogFailureMaxDelayHours >= 1
```

Если в проекте уже есть options validation, использовать её.

Logging

Добавить структурированные aggregated logs.

Start

```
Price collection started. RunId={RunId}; Limit={Limit}
```

Selection

```
Catalog products selected. RunId={RunId}; Selected={Selected};
WorkerId={WorkerId}
```

Queue seed

```
Price collection queue seeded. RunId={RunId}; Selected={Selected};  
Enqueued={Enqueued}
```

Completion

```
Price collection completed. RunId={RunId}; Selected={Selected};  
Enqueued={Enqueued}; Succeeded={Succeeded}; Retry={Retry}; Dead={Dead}
```

Failure

```
Price collection failed. RunId={RunId}; ErrorCode={ErrorCode}
```

Не логировать все URL на уровне Information.

Per-item success logs допускаются только на Debug.

Per-item errors допускаются на Warning.

Tests

Добавить unit, repository integration и end-to-end integration tests.

Unit tests: oldest-first

1. Never checked first

```
GetDueProductsAsync_NeverCheckedItems_AreReturnedFirst
```

2. Oldest checked first

```
GetDueProductsAsync_CheckedItems_AreOrderedByOldestLastCheckedAt
```

3. Stable ID ordering

```
GetDueProductsAsync_SameTimestamps_AreOrderedById
```

4. Inactive excluded

GetDueProductsAsync_InactiveItems_AreExcluded

5. Future next check excluded

GetDueProductsAsync_NextCheckInFuture_IsExcluded

6. Due next check included

GetDueProductsAsync_NextCheckReached_IsIncluded

7. Limit respected

GetDueProductsAsync_LimitIsRespected

Unit tests: use case

8. Successful collection

ExecuteAsync_Success_SelectsEnqueuesProcessesAndFinishesRun

Проверить порядок:

```
crawler start
ingestion start
catalog select
queue enqueue
queue drain
ingestion finish
crawler finish
```

9. No discovery call

ExecuteAsync_DoesNotInvokeProductUrlDiscovery

Новый use case вообще не должен зависеть от discovery service.

10. MaxProductsPerRun

ExecuteAsync_UsesMaxProductsPerRunAsSelectionLimit

11. Empty selection

ExecuteAsync_NoDueProducts_CompletesSuccessfullyWithoutQueue

12. One queue item per catalog item

ExecuteAsync_SelectedProducts_EnqueuesMatchingCatalogIds

13. Success marks catalog checked

ProcessItem_Success_UpdatesLastCheckedNextCheckAndResetsErrors

14. Success updates metadata

ProcessItem_Success_UpdatesExternalIdAndSlugWhenPresent

15. Empty metadata does not erase

ProcessItem_Success_NullMetadataDoesNotEraseCatalogValues

16. Retry does not mark final failure

ProcessItem_TransientFailureWithRetry_DoesNotIncrementCatalogErrors

17. Dead increments errors

ProcessItem_FinalFailure_IncrementsCatalogErrorsAndSchedulesNextCheck

18. Cancellation rethrown

ExecuteAsync_Cancelled_RethrowsOperationCanceledException

19. Run statuses

ExecuteAsync_DeadItems_FinishesRunWithError

20. All success status

```
ExecuteAsync_AllItemsSucceeded_FinishesRunWithOk
```

Unit tests: backoff

21. First failure

```
CatalogRetryPolicy_FirstFailure_ReturnsBaseDelay
```

22. Exponential growth

```
CatalogRetryPolicy_ConsecutiveFailures_UsesExponentialDelay
```

23. Maximum cap

```
CatalogRetryPolicy_DelayDoesNotExceedMaximum
```

24. Overflow protection

```
CatalogRetryPolicy_LargeFailureCount_DoesNotOverflow
```

PostgreSQL integration tests

1. Oldest-first query

Создать записи:

```
A: never checked  
B: checked 10 days ago  
C: checked 1 day ago  
D: inactive  
E: next_check_at tomorrow
```

Ожидаемый результат:

A
B
C

D и **E** не возвращаются.

2. Concurrent reservation

GetDueProductsAsync_ConcurrentWorkers_DoNotReceiveSameCatalogItems

Запустить два параллельных repository calls.

Проверить:

- пересечение IDs пустое;
 - суммарное количество не превышает доступные due rows;
 - используется lease.
-

3. Expired lease

GetDueProductsAsync_ExpiredReservation_BecomesAvailableAgain

4. Active lease

GetDueProductsAsync_ActiveReservation_IsNotSelected

5. Mark checked

MarkCheckedAsync_UpdatesSchedulingStateAndClearsReservation

Проверить:

- `last_checked_at`;
 - `next_check_at`;
 - `consecutive_errors = 0`;
 - reservation cleared;
 - metadata updated.
-

6. Mark failed

```
MarkFailedAsync_IncrementsErrorsSchedulesRetryAndClearsReservation
```

7. Queue linkage

```
PriceCollectQueue_StoresProductCatalogId
```

8. FK enforcement

```
PriceCollectQueue_InvalidProductCatalogId_IsRejected
```

9. Full successful flow

```
CollectProductPricesUseCase_RealDatabase_SavesObservationAndUpdatesCatalog
```

Проверить:

- `crawler_run`;
- `ingestion_run`;
- `queue row`;
- `product`;
- `price snapshot`;
- `catalog last_checked_at`;
- `catalog next_check_at`;
- `consecutive_errors = 0`.

10. Full failed flow

```
CollectProductPricesUseCase_FinalFailure_StoresErrorAndUpdatesCatalogBackoff
```

11. No category requests

Integration test с fake discovery/page loader должен подтвердить, что `collect-prices` не обращается к category discovery.

Schema compatibility

Все schema changes должны быть идемпотентны.

Проверить:

- существующая БД обновляется без потери данных;
- пустая БД создаётся корректно;
- повторный schema bootstrap не падает;
- nullable `product_catalog_id` не ломает старые queue rows;
- существующие routines продолжают работать либо корректно мигрированы.

Documentation

Обновить:

```
docs/architecture.md
README.md
```

Добавить flow

```
Weekly:
catalog-refresh
→ product_catalog

Daily:
collect-prices
→ oldest-first catalog selection
→ queue
→ product / price_snapshot
```

Описать oldest-first

```
1. last_checked_at is null
2. oldest last_checked_at
3. id as stable tie-breaker
```

Описать настройки

```
MaxProductsPerRun
CatalogLeaseSeconds
```

```
SuccessfulCheckIntervalHours
CatalogFailureBaseDelayMinutes
CatalogFailureMaxDelayHours
```

Добавить пример запуска

```
dotnet run --project VarPrice.Worker -- collect-prices
```

Добавить SQL-проверки

```
select
    id,
    normalized_url,
    last_checked_at,
    next_check_at,
    consecutive_errors,
    reserved_until
from product_catalog
order by last_checked_at nulls first, id
limit 50;
```

Non-goals

В рамках MPC-63 запрещено реализовывать:

- новый discovery;
- повторный обход 471 категории внутри price collection;
- деактивацию отсутствующих товаров;
- `is_active = false` по результатам catalog refresh;
- full scheduler;
- cron;
- cloud deployment;
- UI каталога;
- Web API каталога;
- динамические интервалы по популярности;
- отдельную частоту для акционных товаров;
- приоритеты категорий;
- distributed queue;
- Kafka;
- RabbitMQ;
- Redis;
- многосетевой routing Varus/ATB;
- удаление старого `RunCrawlerUseCase` ;
- очистку истории цен;
- bulk HTTP;

- изменение HTML parser вне необходимой интеграции;
- увеличение RPS без отдельного анализа;
- автоматический запуск 40 000 товаров в tests.

Important design decisions

1. Price collection больше не запускает discovery

Рабочий ежедневный режим читает только `product_catalog`.

2. `MaxProductsPerRun` ограничивает batch

```
product_catalog due count может быть 40000
MaxProductsPerRun = 5000
→ выбрано максимум 5000
```

3. Oldest-first исключает постоянную обработку первых URL

Товары ротируются по `last_checked_at`.

4. Lease защищает от параллельных запусков

Два Worker не должны собирать цену одного товара одновременно.

5. Queue retry и catalog backoff — разные уровни

```
Queue retry
→ повтор текущей обработки в рамках run

Catalog next_check_at
→ когда товар снова доступен для нового run
```

Их нельзя смешивать.

6. Успешная карточка сбрасывает error streak

```
consecutive_errors = 0
```

7. История цен сохраняется через существующий observation flow

Не создавать новый параллельный механизм записи цен.

Acceptance criteria

Задача считается выполненной, если:

- [] добавлен `ICollectProductPricesUseCase` ;
- [] добавлен `CollectProductPricesUseCase` ;
- [] добавлен `CollectProductPricesResult` ;
- [] добавлен oldest-first repository method;
- [] never-checked товары выбираются первыми;
- [] затем выбираются самые старые `last_checked_at` ;
- [] inactive товары не выбираются;
- [] future `next_check_at` не выбирается;
- [] выбор ограничивается `MaxProductsPerRun` ;
- [] concurrent workers не получают одинаковые catalog IDs;
- [] добавлен lease;
- [] expired lease переиспользуется;
- [] `crawler_run` создаётся до selection;
- [] `ingestion_run` создаётся для price collection;
- [] selected rows enqueue с `product_catalog_id` ;
- [] queue schema содержит FK на `product_catalog` ;
- [] новый use case не вызывает discovery;
- [] successful item сохраняет observation;
- [] successful item обновляет catalog scheduling state;
- [] successful item сбрасывает `consecutive_errors` ;
- [] transient queue retry не увеличивает catalog error counter;
- [] final dead увеличивает error counter;
- [] final dead выставляет catalog backoff;
- [] reservations очищаются после финального результата;
- [] empty selection завершается со статусом `ok` ;
- [] Worker поддерживает `collect-prices` ;
- [] `catalog-refresh` продолжает работать;
- [] добавлены unit tests;
- [] добавлены concurrency integration tests;
- [] добавлены PostgreSQL integration tests;
- [] schema bootstrap идемпотентен;
- [] документация обновлена;
- [] существующие тесты проходят;
- [] solution собирается без новых warnings;
- [] unrelated code не рефакторился.

Required verification

Codex обязан выполнить:

```
dotnet restore
dotnet build --no-restore
dotnet test --no-build
```

Запустить PostgreSQL integration infrastructure проекта.

Выполнить ручной сценарий.

Шаг 1. Обновить каталог

```
dotnet run --project VarPrice.Worker -- catalog-refresh
```

Шаг 2. Запустить сбор небольшой порции

Настройки:

```
{
  "Crawler": {
    "MaxProductsPerRun": 10,
    "MaxConcurrency": 1,
    "RequestsPerSecond": 0.5,
    "UseStubProductCardExtractor": true,
    "SuccessfulCheckIntervalHours": 24,
    "CatalogLeaseSeconds": 1800
  }
}
```

Запуск:

```
dotnet run --project VarPrice.Worker -- collect-prices
```

Шаг 3. Проверить run

```
select
  id,
  source,
  started_at,
  finished_at,
  status,
  note
from crawler_run
order by id desc
limit 5;
```

Шаг 4. Проверить очередь

```
select
    id,
    run_id,
    product_catalog_id,
    status,
    attempt,
    max_attempts
from price_collect_queue
where run_id = :run_id
order by id;
```

Шаг 5. Проверить каталог

```
select
    id,
    normalized_url,
    last_checked_at,
    next_check_at,
    consecutive_errors,
    reserved_at,
    reserved_until,
    reserved_by
from product_catalog
where id in (
    select product_catalog_id
    from price_collect_queue
    where run_id = :run_id
)
order by id;
```

Ожидается:

```
last_checked_at is not null
next_check_at is not null
consecutive_errors = 0
reservation fields are null
```

Шаг 6. Повторный запуск

Сразу повторно выполнить:

```
dotnet run --project VarPrice.Worker -- collect-prices
```


При наличии достаточного каталога должны быть выбраны следующие oldest-first товары, а не те же 10.

Required production-oriented manual check

После stub-проверки выполнить небольшой реальный запуск:

```
{
  "Crawler": {
    "MaxProductsPerRun": 5,
    "MaxConcurrency": 1,
    "RequestsPerSecond": 0.5,
    "UseStubProductCardExtractor": false
  }
}
```

Подтвердить:

- реальные карточки прочитаны;
- snapshots созданы;
- catalog rows обновлены;
- ошибки отражаются в crawl errors;
- Worker завершается корректно.

Не запускать полный каталог в рамках проверки Codex.

Codex final response format

В финальном ответе Codex обязан предоставить следующие разделы.

1. Summary

Кратко описать новый price collection flow.

2. Changed files

Перечислить все изменённые и добавленные файлы с назначением.

3. Runtime flow

Показать фактический порядок:

```
crawler_run_start
→ ingestion_run_start
```

- catalog oldest-first reservation
- queue enqueue
- queue processing
- snapshot persistence
- catalog scheduling update
- ingestion_run_finish
- crawler_run_finish

4. Oldest-first behavior

Подтвердить:

- never checked first;
- oldest checked next;
- stable ID tie-breaker;
- inactive/future rows excluded.

5. Concurrency protection

Описать:

- lease fields;
- `FOR UPDATE SKIP LOCKED` или эквивалент;
- поведение expired lease;
- результат concurrency test.

6. Queue/catalog integration

Указать:

- как хранится `product_catalog_id`;
- как success обновляет каталог;
- как retry влияет на каталог;
- как dead влияет на каталог.

7. Database changes

Перечислить:

- новые колонки;
- indexes;
- foreign keys;
- routines;
- изменённые routines.

8. Tests

Указать:

- unit tests;
- PostgreSQL integration tests;
- concurrency tests;
- фактический `dotnet build`;
- фактический `dotnet test`;
- результат stub manual run;
- результат real extractor small run.

9. Verification evidence

Привести фактические агрегированные значения:

```
RunId:  
Selected:  
Enqueued:  
Succeeded:  
Retry:  
Dead:  
Catalog rows updated:  
Snapshots created:
```

10. Risks and follow-up

Явно указать:

- деактивация исчезнувших товаров ещё не реализована;
- полноценный scheduler ещё не реализован;
- интервалы пока одинаковы для всех товаров;
- старый `RunCrawlerUseCase` пока сохранён.

Не заявлять об успешном тестировании, если команды фактически не запускались.